RES POLYTECHNIC, BANGALORE-91

CIE Theory Test III - Model Answers

Programming Concepts in Mechanical Engineering | Course Code 25ME34I | Marks 50 | Semester III

SECTION I

Q1 a) Student list – append, remove, sort (7 marks)

```
students = ["Ravi", "Anu", "Kiran", "Meena", "Suresh"]
students.append("Divya")       # add one student
students.remove("Kiran")       # remove one student
students.sort()                # sort
print("Sorted list:", students)
```

Output: `Sorted list: ['Anu', 'Divya', 'Meena', 'Ravi', 'Suresh']`

Q1 b) Count words in data.txt with try, except, finally (6 marks)

```
try:
    f = open("data.txt", "r")
    text = f.read()
    words = text.split()
    print("Number of words =", len(words))
except FileNotFoundError:
    print("Error: data.txt not found")
finally:
    print("Program finished")      # always executes
```

- `try` – code that may cause an error; `except` – runs if the file is missing; `finally` – runs always.

Q1 c) Class Motor (6 marks)

```
class Motor:
    def __init__(self, model, power, colour):
        self.model = model
        self.power = power
        self.colour = colour

    def start(self):
        print(self.model, "motor started")

    def stop(self):
        print(self.model, "motor stopped")

    def display(self):
        print("Model :", self.model)
        print("Power :", self.power, "kW")
        print("Colour:", self.colour)

m1 = Motor("M100", 5.5, "Blue")
m1.display()
m1.start()
m1.stop()
```

Q1 d) DC motor control with L298N and Raspberry Pi (6 marks)

- L298N is an H-bridge driver. The Pi's GPIO pins give only low current (3.3 V), so the driver supplies the higher current/voltage to the motor.
- **IN1/IN2** set the direction; **ENA** enables the motor (or controls speed by PWM).

GPIO connections:

Raspberry Pi	L298N
GPIO 17	IN1
GPIO 27	IN2
GPIO 22	ENA
GND	GND (common ground)

Motor connects to OUT1/OUT2; external supply (e.g. 12 V) connects to the 12V terminal.

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
```

```
GPIO.setup([17, 27, 22], GPIO.OUT)

GPIO.output(17, GPIO.HIGH)    # forward
GPIO.output(27, GPIO.LOW)
GPIO.output(22, GPIO.HIGH)    # enable
time.sleep(5)                 # run for 5 s

GPIO.output(22, GPIO.LOW)     # stop
GPIO.cleanup()
```

OR

Q2 a) Student records using a dictionary (7 marks)

```
students = {"S1": {"name": "Ravi", "age": 18, "grade": "A"}}

def add_student(sid, name, age, grade):
    students[sid] = {"name": name, "age": age, "grade": grade}

def remove_student(sid):
    if sid in students:
        del students[sid]
    else:
        print("Record not found")

add_student("S2", "Anu", 17, "B")
add_student("S3", "Kiran", 18, "A")
remove_student("S1")

for sid, rec in students.items():
    print(sid, rec["name"], rec["age"], rec["grade"])
```

Q2 b) Write and read random sensor readings; file modes (6 marks)

```
import random

with open("sensor.txt", "w") as f:         # write
    for i in range(10):
        f.write(str(round(random.uniform(20, 40), 2)) + "\n")

with open("sensor.txt", "r") as f:         # read
    for line in f:
        print(line.strip())
```

Mode	Purpose
"w"	Write; creates the file, erases old content
"r"	Read (default); file must exist
"a"	Append; adds new data at the end without erasing

Q2 c) Inheritance and method overriding (6 marks)

```
class Machine:
    def operate(self):
        print("Machine is operating")

class Lathe(Machine):
    def operate(self):
        print("Lathe is turning the workpiece")

class Drill(Machine):
    def operate(self):
        print("Drill is making a hole")

for m in (Lathe(), Drill()):
    m.operate()
```

- **Inheritance:** Lathe and Drill acquire the properties of Machine.
- **Overriding:** a derived class defines a method with the same name as the base class; when called on the derived object, the derived version runs instead of the base version.

Q2 d) DHT11 and PIR sensors with Raspberry Pi; cooling fan (6 marks)

- **DHT11:** 3 pins – VCC (3.3 V), GND, DATA (to a GPIO pin, e.g. GPIO 4, with a pull-up resistor). It gives digital temperature and humidity data.
- **PIR:** 3 pins – VCC (5 V), GND, OUT (to a GPIO input, e.g. GPIO 17). OUT goes HIGH when motion is detected.

- Fan connects to a GPIO pin (e.g. GPIO 18) through a relay/transistor driver.

```
import RPi.GPIO as GPIO
import Adafruit_DHT
import time

GPIO.setmode(GPIO.BCM)
GPIO.setup(18, GPIO.OUT)            # fan

while True:
    hum, temp = Adafruit_DHT.read_retry(Adafruit_DHT.DHT11, 4)
    if temp is not None and temp > 35:
        GPIO.output(18, GPIO.HIGH)  # fan ON
    else:
        GPIO.output(18, GPIO.LOW)   # fan OFF
    time.sleep(2)
```

SECTION II

Q3 a) X-bar and R chart (7 marks)

```
import numpy as np
import matplotlib.pyplot as plt

data = np.array([[25.0, 25.1, 24.9, 25.0],
                 [25.2, 25.0, 25.1, 24.9],
                 [24.8, 25.0, 25.1, 25.0],
                 [25.1, 25.3, 25.0, 25.2],
                 [24.9, 25.0, 24.8, 25.1]])

xbar = data.mean(axis=1)
R = data.max(axis=1) - data.min(axis=1)
print("X-bar:", xbar)
print("R    :", R)

xbb, Rb = xbar.mean(), R.mean()
A2, D3, D4 = 0.729, 0, 2.282            # constants for n = 4

plt.figure(figsize=(8, 6))
plt.subplot(2, 1, 1)
plt.plot(xbar, marker='o')
plt.axhline(xbb, color='g')
plt.axhline(xbb + A2*Rb, color='r', linestyle='--')
plt.axhline(xbb - A2*Rb, color='r', linestyle='--')
plt.title("X-bar chart")

plt.subplot(2, 1, 2)
plt.plot(R, marker='o')
plt.axhline(Rb, color='g')
plt.axhline(D4*Rb, color='r', linestyle='--')
plt.axhline(D3*Rb, color='r', linestyle='--')
plt.title("R chart")
plt.tight_layout()
plt.show()
```

(Data are assumed; formulas: X-bar = mean of sample, R = max – min; UCL = $\bar{\bar{X}}$ + A2·$\bar{R}$, LCL = $\bar{\bar{X}}$ – A2·$\bar{R}$.)

Q3 b) Mean, variance, standard deviation, Cp and Cpk (6 marks)

```
import numpy as np

d = [25.0, 25.1, 24.9, 25.2, 24.8, 25.0, 25.1, 24.9, 25.0, 25.0]
USL, LSL = 25.5, 24.5

mean = np.mean(d)
var = np.var(d, ddof=1)
sd = np.std(d, ddof=1)

Cp = (USL - LSL) / (6 * sd)
Cpk = min((USL - mean) / (3 * sd), (mean - LSL) / (3 * sd))

print("Mean =", mean)
print("Variance =", var)
print("Std Dev =", sd)
print("Cp =", Cp)
print("Cpk =", Cpk)
```

Formulae: Cp = (USL – LSL) / 6σ; Cpk = min[(USL – μ)/3σ, (μ – LSL)/3σ]. Cpk ≥ 1.33 means the process is capable.

Q3 c) Hypothesis testing; t-test vs ANOVA (6 marks)

Hypothesis testing: a statistical method to decide, from sample data, whether a claim about a process is true.

- **H_0 (null):** no difference/change. **H_1 (alternate):** there is a difference.
- Compute the test statistic and p-value; if $p < 0.05$, reject H_0.

t-test	ANOVA
Compares means of **two** groups	Compares means of **three or more** groups
Example: hardness of components from Batch A and Batch B	Example: hardness of components from Batch A, B and C
Uses t-statistic	Uses F-statistic

```
from scipy import stats
stats.ttest_ind(batchA, batchB)          # t-test
stats.f_oneway(batchA, batchB, batchC)   # ANOVA
```

Q3 d) Finite element method (6 marks)

Steps:

1. Discretisation – divide the body into elements and nodes (mesh).
2. Select the element type and shape (displacement) function.
3. Derive the element stiffness matrix.
4. Assemble into the global stiffness matrix: [K]{u} = {F}.
5. Apply boundary conditions and loads.
6. Solve for nodal displacements.
7. Calculate strains, stresses and reactions.

Applications: stress analysis of machine parts, heat transfer analysis, vibration/crash analysis.

Advantages: handles complex shapes and materials; reduces cost of physical testing (gives accurate results).

OR

Q4 a) p-chart (7 marks)

```
import numpy as np
import matplotlib.pyplot as plt

defects = np.array([4, 3, 5, 2, 6, 4, 3, 7, 2, 4])
n = 50

p = defects / n                              # fraction defective
pbar = defects.sum() / (n * len(defects))   # centre line
sig = np.sqrt(pbar * (1 - pbar) / n)
UCL = pbar + 3 * sig
LCL = max(0, pbar - 3 * sig)

print("p =", p)
print("CL =", pbar, "UCL =", UCL, "LCL =", LCL)

plt.plot(range(1, 11), p, marker='o')
plt.axhline(pbar, color='g', label='CL')
plt.axhline(UCL, color='r', linestyle='--', label='UCL')
plt.axhline(LCL, color='r', linestyle='--', label='LCL')
plt.xlabel("Sample"); plt.ylabel("Fraction defective")
plt.title("p-chart"); plt.legend(); plt.show()
```

Calculation: total defects = 40; $\bar{p}$ = 40/500 = 0.08 (CL); $\sigma = \sqrt{(0.08 \times 0.92/50)} = 0.0384$; UCL = 0.08 + 3(0.0384) = **0.195**; LCL = 0.08 – 0.115 = –0.035 → **0**.

Q4 b) In-control and out-of-control samples (6 marks)

```
avg = [25.0, 25.1, 24.9, 25.5, 25.0, 24.5, 25.2, 25.1]
UCL, LCL = 25.4, 24.6

for i, x in enumerate(avg, start=1):
    if LCL <= x <= UCL:
        print("Sample", i, x, "-> In control")
    else:
        print("Sample", i, x, "-> Out of control")
```

Result: Sample 4 (25.5 > UCL) and Sample 6 (24.5 < LCL) are out of control; all others are in control.

Q4 c) Pareto chart (6 marks)

```
import numpy as np
import matplotlib.pyplot as plt

labels = ["Blowholes", "Cracks", "Misruns", "Shrinkage", "Others"]
count = np.array([40, 25, 15, 10, 10])
cum = np.cumsum(count) / count.sum() * 100

fig, ax = plt.subplots()
ax.bar(labels, count)
ax.set_ylabel("Number of defects")
ax2 = ax.twinx()
ax2.plot(labels, cum, color='r', marker='o')
ax2.set_ylabel("Cumulative %")
plt.title("Pareto chart")
plt.show()
```

Cumulative %: 40, 65, 80, 90, 100. **Use in continuous improvement:** it follows the 80/20 rule – blowholes, cracks and misruns cause 80% of defects, so improvement effort is focused on these "vital few" first, and progress is checked by drawing the chart again.

Q4 d) Bar element stiffness matrix and assembly (6 marks)

For one bar element (area A, length L, modulus E):

$$[k] = \frac{AE}{L}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

For a two-element bar (nodes 1-2-3), with k1 = A1E1/L1 and k2 = A2E2/L2:

$$[K] = \begin{bmatrix} k1 & -k1 & 0 \\ -k1 & k1+k2 & -k2 \\ 0 & -k2 & k2 \end{bmatrix}$$

Element 1 occupies rows/columns 1-2 and element 2 occupies 2-3; the overlapping term at node 2 is added (k1 + k2).

Properties: (1) symmetric matrix; (2) singular before boundary conditions are applied (determinant = 0); (3) diagonal terms are positive (any two).